

Assignment 2 — Solution Key

Number Systems, Representation, and Arithmetic Hardware

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

2.1 Conceptual

Q1. Sign-magnitude: negation flips only the sign bit; the defect is that it produces *two zeros* (+0 and -0), and addition must inspect signs and magnitudes and route the smaller from the larger — a subtractor as well as an adder is needed, so it is awkward for a single-adder ALU. **One's complement:** negation flips all bits; the defect is again *two zeros*, and addition requires an *end-around carry* (the carry-out must be wrapped back into the least-significant bit) — extra hardware and a special case. **Two's complement:** negation is flip-all-bits-then-add-1, there is *exactly one zero*, and plain binary addition of two's-complement operands just works, with the carry-out simply discarded: $-x$ is defined so that $x + (-x) = 0$ in the same width. The ALU therefore needs only an adder — no sign inspection, no end-around carry — which is why a hardware designer picks it.

Q2. Overflow occurs *exactly when the result's sign is wrong*: the two operands share a sign bit and the result's sign bit differs (adding two positives yields a negative, or two negatives yield a positive). Carry-out alone is not the test. *Carry-out, correct result:* $+7 + (-3)$ in 8-bit two's complement ($0000\ 0111 + 1111\ 1101 = 1\ 0000\ 0100$); the carry-out of the MSB is discarded and $0000\ 0100 = +4$ is correct — carry-out occurred, no overflow. *No carry-out, overflow:* $+100 + +50$ in 8-bit two's complement ($0110\ 0100 + 0011\ 0010 = 1001\ 0110$); there is no carry-out, yet the result reads as -106 — a wrong sign for the sum of two positives, so overflow.

2.2 Numerical

Q3. (a) Repeated division by 2:

$$89/2 = 44\ r\ 1; \quad 44/2 = 22\ r\ 0; \quad 22/2 = 11\ r\ 0; \quad 11/2 = 5\ r\ 1;$$

$$5/2 = 2\ r\ 1; \quad 2/2 = 1\ r\ 0; \quad 1/2 = 0\ r\ 1.$$

Reading remainders bottom-up: $89_{10} = 1011001_2$. (b) Grouping the 7 bits into nibbles from the right: $0101\ 1001_2 = 59_{16}$. Check: $5 \times 16 + 9 = 80 + 9 = 89$. (c) $+36 = 0010\ 0100_2$. Flip: $1101\ 1011$; add 1: $1101\ 1100$. So $-36 = 1101\ 1100_2$. Verify: $1101\ 1100 + 0010\ 0100 = 1\ 0000\ 0000$; discard the carry to get $0000\ 0000 = 0$, confirming $36 + (-36) = 0$.

Q4. (a) $6.25 = 0000\ 0110.0100\ 0000$ (i.e. $6 + 0.25$); $1.5 = 0000\ 0001.1000\ 0000$. Adding bitwise:

$$0000\ 0110.0100\ 0000 + 0000\ 0001.1000\ 0000 = 0000\ 0111.1100\ 0000,$$

which reads $7.75 = 6.25 + 1.5$. (b) $-2 = 1111\ 1110$ (8-bit two's complement). Add the integer parts: $0000\ 0110 + 1111\ 1110 = 1\ 0000\ 0100$; discard the carry to get $0000\ 0100$, i.e. integer part 4;

the fraction 0.25 is untouched: 4.25. Ordinary integer addition with the carry-out discarded gives the correct signed fixed-point result — no special subtraction hardware needed.

2.3 Design

Q5. $+5 = 0101$; -2 : flip $0010 \rightarrow 1101$, add $1 \Rightarrow 1110$. Add with a 4-bit ripple-carry adder, $C_{in} = 0$:

$$\text{bit 0: } 1 + 0 + 0 = 1, C = 0$$

$$\text{bit 1: } 0 + 1 + 0 = 1, C = 0$$

$$\text{bit 2: } 1 + 1 + 0 = 0, C = 1$$

$$\text{bit 3: } 0 + 1 + 1 = 0, C = 1$$

Result $0011 = +3$, carry-out = 1. The result is **correct**: $+5 + (-2) = +3$, and the carry-out is simply discarded. Since both operands have sign bit 0 and 1 respectively (opposite signs), the wrong-sign test cannot trigger — no overflow; the carry-out is just the 5th bit of the magnitude sum and is irrelevant to the two's-complement result.

Q6. Since $A - B = A + (-B)$ and $-B$ is “flip B and add 1,” the ALU uses a single adder with a **MUX** on the B input selecting either B (for ADD) or $\bar{B}$ (flipped B , for SUB), and a forced **carry-in** on the LSB:

Operation	MUX selects	Carry-in	Adder computes
ADD	B	0	$A + B$
SUB	$\bar{B}$	1	$A + \bar{B} + 1 = A - B$

No separate subtractor is needed: the same adder handles both, the MUX and the forced carry-in being the only added logic. (Compare P&H, Sec. 3.2, p.188: subtraction is implemented by inverting the subtrahend and adding 1.)